

5.4 NTP Time ESP32

ESP32 NTP Time Synchronization (STA Mode)



Note: This tutorial explains how to get accurate time from the internet using NTP (Network Time Protocol) via WiFi. This is a common need for ESP32 — the device has no clock battery, so time is lost on power off and must be re-synced every boot. Please complete the STA mode (WiFi connection) tutorial first.

Why Do We Need NTP (Network Time Protocol)?

In daily use, you may encounter these scenarios:

- ▶ **Data logging:** Sensor data needs timestamps; without time, you don't know when data was collected
- ▶ **Scheduled triggers:** Automatically turn on lights at 7 AM — you need to know "what time is it now"
- ▶ **Debug logging:** When device has issues, you need to know "what happened when"
- ▶ **Certificate validation:** HTTPS connections depend on correct system time to validate certificate validity

But ESP32 has a **fundamental problem**: It has no battery-backed clock (RTC). Time is lost on power off, and every boot starts from an old default value (could be 1970 or January 1, 2024).

Simple analogy: ESP32 is like a person without a watch — every time they wake up, they don't know what time it is. NTP is the mechanism to "ask the internet server: what time is it?". As long as you connect to WiFi, you can get accurate time from the internet.

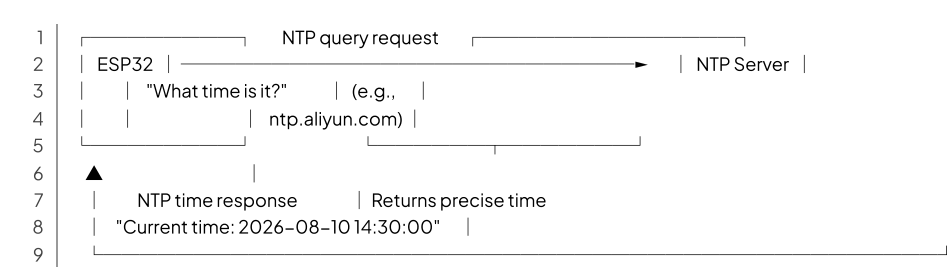
This tutorial's goal: ESP32 connects to WiFi → gets time from NTP server → sets timezone → prints current time to serial every second.

Related Knowledge

1. What is NTP?

NTP (Network Time Protocol) is a protocol for synchronizing computer system clocks over a network. There are thousands of NTP servers worldwide that anyone can query for free.

Workflow:



2. ESP32 Has No Clock Battery

Unlike computers and phones, ESP32 **has no battery-backed hardware clock (RTC)**. This means:

- ▶ Time is completely lost on power off
- ▶ Every boot must re-sync NTP
- ▶ Before sync, time is invalid (may show 1970 or 2024)
- ▶ **Time can only be used after NTP sync succeeds**



Key understanding: NTP sync is not instantaneous! After ESP32 sends a request, it must wait for the server response before time becomes valid. This can take 1–10 seconds depending on network conditions.

3. What is a Timezone?

A **Timezone** is a time standard for different regions on Earth. The world is divided into 24 timezones, each differing by 1 hour.

Simple analogy: When Beijing is 3 PM, London is 7 AM and New York is 2 AM. This is because they're in different timezones.

ESP32's default time from NTP is **UTC (Coordinated Universal Time)**, and we need to set the timezone to see local time.

4. POSIX Timezone String

[Back to Elegoo](#)

ESP32 uses POSIX standard timezone strings to set the timezone. Format:

1 | StandardTimezoneName[UTCOffset][DaylightSaving[offset,start,end]]

Common timezone strings:

Region	TZ String	Meaning
China	CST-8	UTC+8, no DST
Japan	JST-9	UTC+9, no DST
UK (winter)	GMT0	UTC+0, no DST
US Eastern	EST+5	UTC-5, no DST
US Pacific	PST+8PDT,M3.2.0,M11.1.0	UTC-7 with DST Mar-Nov

Meaning of each part (using CST-8 as example):

- ▶ CST : Timezone name (China Standard Time)
- ▶ -8 : Difference from UTC. **Note the sign direction:** -8 means UTC+8 (East 8), +5 means UTC-5 (West 5)



Beginner pitfall: The sign is easy to get backwards! China is UTC+8, but the TZ string uses CST-8 (negative sign). Think of it this way: UTC time plus the offset equals local time, so CST-8 means "local time = UTC + 8 hours".

5. Common NTP Servers

Server Address	Location	Notes
ntp.aliyun.com	China	Alibaba Cloud NTP node, recommended for China, fast
pool.ntp.org	Global	Global public NTP, may be blocked or slow
time.windows.com	Global	Microsoft NTP server
ntp.tencent.com	China	Tencent NTP node, China alternative

6. ESP32 Time Functions

ESP32 uses standard C library time functions:

Function	Description
<code>configTime(gmtOffset, dstOffset, server1, server2)</code>	Configure NTP server (ESP32 specific)
<code>setenv("TZ", tz_string, 1)</code>	Set timezone environment variable
<code>tzset()</code>	Apply timezone setting (must be called after <code>setenv</code>)
<code>time(&t)</code>	Get current time (seconds since Jan 1, 1970)
<code>localtime_r(&t, &tm)</code>	Convert seconds to readable time structure

Key fields of `tm` structure:

Field	Meaning	Range
<code>tm_year</code>	Year (from 1900; 2024 has value 124)	0+

Field	Meaning	Range
<code>tm_mon</code>	Month (0 = January)	0–11
<code>tm_mday</code>	Day	1–31
<code>tm_hour</code>	Hour	0–23
<code>tm_min</code>	Minute	0–59
<code>tm_sec</code>	Second	0–59
<code>tm_wday</code>	Day of week (0 = Sunday)	0–6



Beginner pitfall: `tm_year` needs +1900 for actual year (2024 = `tm_year` 124), `tm_mon` needs +1 for actual month (January = `tm_mon` 0).

Library Installation Guide

This tutorial uses ESP32's built-in `WiFi.h` and `time.h`, no third-party libraries needed.

If you haven't installed ESP32 board support yet, please refer to "Library Installation Guide" in Part 1 "WiFi Connection Tutorial".

Key Functions

1. `configTime()` — Configure NTP Server

```
1 | configTime(gmtOffset_sec, daylightOffset_sec, server1, server2);
```

Parameter	Description
<code>gmtOffset_sec</code>	UTC offset in seconds. Set to 0 when using TZ string
<code>daylightOffset_sec</code>	Daylight saving offset in seconds. Set to 0
<code>server1</code>	Primary NTP server address (string)
<code>server2</code>	Backup NTP server address (optional)

Example:

```
1 | configTime(0, 0, "ntp.aliyun.com", "pool.ntp.org");
```



Note: Set `gmtOffset_sec` to 0 here because timezone is controlled by TZ string, not in `configTime`.

2. `setenv()` + `tzset()` — Set Timezone

```
1 | setenv("TZ", "CST-8", 1); // Third parameter 1 means overwrite existing
2 | tzset();                // Must be called to take effect
```

- ▶ `setenv("TZ", "CST-8", 1)` : Sets timezone environment variable to China Standard Time
- ▶ `tzset()` : Tells system "re-read timezone setting"



Order matters: Must `setenv` first, then `tzset`, and after `configTime`.

3. `time()` — Get Current Timestamp

```
1 | time_t now;  
2 | time(&now); // Store current timestamp in now
```

[Back to Elegoo](#)

Returns seconds from Jan 1, 1970 00:00:00 (UTC) to now. This value isn't intuitive by itself, needs `localtime_r` to convert.

4. `localtime_r()` — Convert to Readable Time

```
1 | struct tm timeinfo;  
2 | localtime_r(&now, &timeinfo);  
3 | // timeinfo.tm_year = 124 (actual year 2024)  
4 | // timeinfo.tm_mon = 7 (actual month August)  
5 | // timeinfo.tm_mday = 10 (10th)  
6 | // timeinfo.tm_hour = 14 (14:00)  
7 | // timeinfo.tm_min = 30 (30 minutes)  
8 | // timeinfo.tm_sec = 45 (45 seconds)  
9 | // timeinfo.tm_wday = 6 (Saturday)
```

Why `localtime_r` instead of `localtime` ? `localtime` returns pointer to static memory, unsafe (multiple calls overwrite). `localtime_r` writes to your provided structure, safer.

Complete Code Analysis (By Module)

Reading Guide: Code is divided into **4 modules** by function. Each module handles one independent category. Recommended to read in order.

 [DOWNLOAD](#)

Module 1: Configuration

Responsibility: Centralizes all adjustable parameters including WiFi credentials, NTP server, timezone, etc.

```
1 | // --- WiFi Credentials ---  
2 | const char* WIFI_SSID = "YourWiFi";  
3 | const char* WIFI_PASS = "YourPassword";  
4 |  
5 | // --- NTP Server ---  
6 | const char* NTP_SERVER_1 = "ntp.aliyun.com";  
7 | const char* NTP_SERVER_2 = "pool.ntp.org";  
8 |  
9 | // --- Timezone ---  
10 | const char* TZ_INFO = "CST-8";  
11 |  
12 | // --- Pin & Timing ---  
13 | #define LED_BUILTIN 2  
14 | const unsigned long WIFI_TIMEOUT_MS = 15000;  
15 | const unsigned long NTP_SYNC_TIMEOUT = 10000;  
16 | const unsigned long DISPLAY_INTERVAL = 1000;
```

Code Explanation:

Parameter	Meaning	Adjustment Suggestion
WIFI_SSID / WIFI_PASS	WiFi credentials	Change to yours
NTP_SERVER_1	Primary NTP server	ntp.aliyun.com recommended for China
NTP_SERVER_2	Backup NTP server	Can set to empty string "" for single server
TZ_INFO	Timezone string	Modify based on your location
WIFI_TIMEOUT_MS	WiFi connect timeout (15 sec)	Increase for weak signal
NTP_SYNC_TIMEOUT	NTP sync timeout (10 sec)	Increase for slow network

Parameter	Meaning	Adjustment Suggestion
<code>DISPLAY_INTERVAL</code>	Time display interval (1 sec)	Can change to 5 sec if too frequent



Must modify: Change `WIFI_SSID` and `WIFI_PASS` to your WiFi info. If in China, recommend setting `NTP_SERVER_1` to `ntp.aliyun.com`.

Module 2: WiFi Connection

Responsibility: Connect to WiFi router and wait for success. This is prerequisite for NTP sync — network must exist first.

```

1  bool connectWiFi() {
2      Serial.print("[WiFi] Connecting to ");
3      Serial.println(WIFI_SSID);
4
5      WiFi.mode(WIFI_STA);
6      WiFi.begin(WIFI_SSID, WIFI_PASS);
7
8      unsigned long start = millis();
9      while (WiFi.status() != WL_CONNECTED && millis() - start < WIFI_TIMEOUT_MS) {
10         delay(500);
11         Serial.print(".");
12     }
13
14     if (WiFi.status() == WL_CONNECTED) {
15         Serial.println("\n[WiFi] Connected!");
16         Serial.print("[WiFi] IP: ");
17         Serial.println(WiFi.localIP());
18         Serial.print("[WiFi] RSSI: ");
19         Serial.print(WiFi.RSSI());
20         Serial.println(" dBm");
21         return true;
22     } else {
23         Serial.println("\n[WiFi] Failed to connect.");
24         return false;
25     }
26 }
```

Logic:

1. `WiFi.mode(WIFI_STA)` : Switch to STA mode
2. `WiFi.begin(...)` : Initiate WiFi connection
3. Poll `WiFi.status()` waiting for connection, check every 500ms, max wait 15 seconds
4. On success, print IP and signal strength, return `true`
5. On timeout, return `false`



Difference from Part 1: Uses non-blocking `while` loop instead of event callbacks. Simpler code, suitable for beginners. If WiFi can't connect, program stops in infinite loop at end of `setup()` (LED fast blink), prompting user to check WiFi info.

Module 3: NTP Synchronization

Responsibility: Configure NTP server and timezone, poll waiting for time sync to succeed.

```

1  bool syncNTP() {
2      // Step 1: Configure NTP server
3      configTime(0, 0, NTP_SERVER_1, NTP_SERVER_2);
4
5      // Step 2: Set timezone
6      setenv("TZ", TZ_INFO, 1);
7      tzset();
8
9      // Step 3: Wait for sync
10     Serial.println("[NTP] Waiting for time synchronization...");
11
12     unsigned long start = millis();
13     time_t now;
14 }
```

```

15 struct tm timeinfo;
16
17 while (millis() - start < NTP_SYNC_TIMEOUT) {
18     time(&now);
19     localtime_r(&now, &timeinfo);
20
21     // Valid synced time has year >= 2024 (tm_year >= 124)
22     if (timeinfo.tm_year >= 124) {
23         Serial.println("[NTP] Time synchronized!");
24         return true;
25     }
26
27     delay(200);
28 }
29
30 Serial.println("[NTP] Sync timeout.");
31 return false;
}

```

Three-step flow:

1. `configTime(0, 0, server1, server2)` : Tells ESP32 which NTP servers to query. First two params set to 0 because timezone is controlled by TZ string.
2. `setenv("TZ", "CST-8", 1) + tzset()` : Sets timezone. `setenv` sets environment variable, `tzset` makes it effective. **Must call in this order, and after `configTime`** .
3. **Poll for sync:** Repeatedly call `time()` + `localtime_r()` to check if time is valid. Before sync, `tm_year` may be 70 (1970) or 124 (Jan 1, 2024); after sync it becomes actual year (e.g., 126 for 2026).



Key check: Use `tm_year >= 124` (i.e., year 2024 or later) as "time synced" criterion. If sync fails, time stays at old value.



Beginner pitfall: NTP sync is **asynchronous**! After calling `configTime()` , ESP32 sends request in background and waits for response. `time()` return value won't immediately become correct — must **poll and wait**.

Module 4: Time Display

Responsibility: Get current time, format to readable string, print to serial.

```

1 void printTime() {
2     time_t now;
3     struct tm timeinfo;
4
5     time(&now);
6     localtime_r(&now, &timeinfo);
7
8     // Format: YYYY-MM-DD HH:MM:SS (Weekday) TZ: CST-8
9     Serial.printf("[Time] %04d-%02d-%02d %02d:%02d:%02d",
10         timeinfo.tm_year + 1900,
11         timeinfo.tm_mon + 1,
12         timeinfo.tm_mday,
13         timeinfo.tm_hour,
14         timeinfo.tm_min,
15         timeinfo.tm_sec);
16
17     // Weekday
18     const char* weekdays[] = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"};
19     Serial.printf(" (%s)", weekdays[timeinfo.tm_wday]);
20
21     // Timezone
22     Serial.printf(" TZ: %s", TZ_INFO);
23
24     Serial.println();
25 }

```

Format Explanation:

`Serial.printf` uses `printf` -style format placeholders:

Placeholder	Meaning	Example
%04d	4-digit integer, zero-padded	2026
%02d	2-digit integer, zero-padded	08 , 10
%s	String	"Sat"

Field Corrections:

- `tm_year + 1900` : Year needs +1900 (because `tm_year` counts from 1900)
- `tm_mon + 1` : Month needs +1 (because `tm_mon` starts from 0)
- `tm_wday` : Use directly (0=Sunday, 6=Saturday)

Output Example:

```
1 | [Time] 2026-08-10 14:30:45 (Sat) TZ: CST-8
```

Main Program

Responsibility: Call modules in order, drive the entire flow.

```

1  void setup() {
2    Serial.begin(115200);
3    delay(1000);
4
5    Serial.println("\n=====");
6    Serial.println(" ESP32 NTP Time Sync Example");
7    Serial.println("=====\\n");
8
9    pinMode(LED_BUILTIN, OUTPUT);
10
11   // Step 1: Connect to WiFi (Module 2)
12   if (!connectWiFi()) {
13     Serial.println("[ERROR] Cannot continue without WiFi.");
14     while (true) {
15       digitalWrite(LED_BUILTIN, !digitalRead(LED_BUILTIN));
16       delay(200);
17     }
18   }
19
20   // Step 2: Sync NTP time (Module 3)
21   if (!syncNTP()) {
22     Serial.println("[WARN] Time sync may have failed. Check NTP server.");
23   }
24
25   // Step 3: Display time every second (Module 4)
26   Serial.println("\\n[Time] Starting periodic time display...\\n");
27   digitalWrite(LED_BUILTIN, HIGH);
28
29   unsigned long lastDisplay = 0;
30   while (true) {
31     unsigned long now = millis();
32     if (now - lastDisplay >= DISPLAY_INTERVAL) {
33       lastDisplay = now;
34       printTime();
35     }
36     delay(10);
37   }
38 }
39
40 void loop() {
41   // Never reached
42 }
```

Flow:

1. **Connect WiFi:** Continue on success, LED fast blink on failure

2. **Sync NTP**: Return on success, warn on timeout (but doesn't block)

3. **Print time every second**: Use `while(true)` loop to continuously call `printTime()`

[Back to Elegoo](#)



Why `while(true)` in `setup()` instead of `loop()` ? This tutorial only demonstrates "get time and print", no other tasks needed. Putting it in `setup()` is more intuitive. To add other features (like sensor control), move time display logic to `loop()`.

Operation Steps Guide

Step 1: Modify Configuration

```
1 | const char* WIFI_SSID = "YourWiFiName";
2 | const char* WIFI_PASS = "YourWiFiPassword";
```

If outside China, also modify:

```
1 | const char* NTP_SERVER_1 = "pool.ntp.org"; // Change to nearby NTP server
2 | const char* TZ_INFO = "EST+5";           // Change to your timezone
```

Step 2: Upload and Open Serial Monitor

1. Connect ESP32 with USB cable
2. Select correct board and port in Arduino IDE
3. Click upload button
4. Open Serial Monitor (baud rate 115200)

Step 3: Observe Output

Normally, you should see:

```
1 | =====
2 | ESP32 NTP Time Sync Example
3 | =====
4 |
5 | [WiFi] Connecting to MyHomeWiFi
6 | .....
7 | [WiFi] Connected!
8 | [WiFi] IP: 192.168.1.105
9 | [WiFi] RSSI: -48 dBm
10 | [NTP] Waiting for time synchronization...
11 | [NTP] Time synchronized!
12 |
13 | [Time] Starting periodic time display...
14 |
15 | [Time] 2026-08-10 14:30:45 (Sat) TZ: CST-8
16 | [Time] 2026-08-10 14:30:46 (Sat) TZ: CST-8
17 | [Time] 2026-08-10 14:30:47 (Sat) TZ: CST-8
18 | ...
```

If WiFi connection fails:

```
1 | [WiFi] Connecting to WrongWiFi
2 | .....
3 | [WiFi] Failed to connect.
4 | [ERROR] Cannot continue without WiFi.
```

LED will fast blink, check if WiFi credentials are correct.

Expected Serial Output

Scenario 1: Normal Operation

```
1 | [WiFi] Connecting to MyHomeWiFi
2 | .....
3 | [WiFi] Connected!
4 |
```

```
5 | [WiFi] IP: 192.168.1.105
6 | [WiFi] RSSI: -45 dBm
7 | [NTP] Waiting for time synchronization...
8 | [NTP] Time synchronized!
9 |
10 | [Time] Starting periodic time display...
11 |
12 | [Time] 2026-08-10 14:30:00 (Sat) TZ: CST-8
13 | [Time] 2026-08-10 14:30:01 (Sat) TZ: CST-8
   | [Time] 2026-08-10 14:30:02 (Sat) TZ: CST-8
```

[Back to Elegoo](#)

Scenario 2: WiFi Connection Failure

```
1 | [WiFi] Connecting to WrongWiFi
2 | .....
3 | [WiFi] Failed to connect.
4 | [ERROR] Cannot continue without WiFi.
5 | (LED flashes rapidly)
```

Scenario 3: NTP Sync Timeout

```
1 | [WiFi] Connected!
2 | [WiFi] IP: 192.168.1.105
3 | [NTP] Waiting for time synchronization...
4 | [NTP] Sync timeout. Using whatever time is available.
5 | [WARN] Time sync may have failed. Check NTP server.
6 |
7 | [Time] Starting periodic time display...
8 |
9 | [Time] 1970-01-01 08:00:00 (Thu) TZ: CST-8
10 | [Time] 1970-01-01 08:00:01 (Thu) TZ: CST-8
```



Scenario 3 means NTP server unreachable, time shows UTC+8 of 1970 (Jan 1, 1970 00:00:00 UTC = Beijing time 08:00:00). Check if NTP server address is correct.

FAQ

Q1: Why does time show 1970 or 2024?

A: NTP sync failed. ESP32 starts time from default value (usually 1970 or Jan 1, 2024) on boot. Check:

- Is WiFi actually connected?
- Is NTP server address correct?
- Can network reach NTP server? (Some routers restrict UDP ports)
- Try a different NTP server

Q2: Time is correct but timezone is wrong?

A: Check if [TZ_INFO](#) is set correctly. For example:

- China: [CST-8](#)
- Japan: [JST-9](#)
- US Eastern: [EST+5](#)

Q3: Why use [ntp.aliyun.com](#) instead of [pool.ntp.org](#) ?

A: [pool.ntp.org](#) often times out or responds slowly in China. Alibaba's NTP server has nodes in China, faster and more stable. If outside China, [pool.ntp.org](#) works fine.

Q4: What does [tm_year >= 124](#) mean in code?

A: [tm_year](#) counts years from 1900. [124](#) represents 1900 + 124 = 2024. Before NTP sync, time may stay at boot default (2024 or earlier); after sync it becomes current year (in 2026, [tm_year = 126](#)). So [>= 124](#) checks if time is valid.

Q5: Does ESP32 need to re-sync after power off/reboot?

A: Yes. ESP32 has no clock battery, time is lost every power off. Must repeat "connect WiFi → sync NTP" flow each time.

Q6: How long does sync take?

A: Usually 1–5 seconds. May take longer if network is slow or NTP server responds slowly. Code sets 10–second timeout, modifiable via `NTP_SYNC_TIMEOUT` in Module 1.

Q7: Can I use multiple NTP servers simultaneously?

A: Yes. `configTime` supports two servers; first is primary, second is backup. If primary is unreachable, ESP32 automatically tries backup.

Q8: How to handle daylight saving time?

A: China doesn't use DST, so `CST-8` is sufficient. In DST regions (like US, Europe), use TZ string with DST, such as `PST+8PDT,M3.2.0,M11.1.0`.

Troubleshooting Table

Problem	Possible Cause	Solution
WiFi won't connect	Wrong credentials	Check case, reconfigure
WiFi won't connect	Weak signal	Move closer to router
NTP sync timeout	NTP server unreachable	Switch to ntp.aliyun.com or check firewall
Time shows 1970	NTP not synced	Check NTP server address
Wrong timezone	TZ_INFO misconfigured	Modify TZ_INFO
Time jumps	NTP correction	Normal, corrects on sync
LED keeps blinking	WiFi connection failed	Check credentials

Modification Suggestions

1. Add Time to OLED Display

If you have an OLED screen, display time simultaneously:

```
1 // After printTime(), add:
2 display.setCursor(0, 0);
3 display.printf("%04d-%02d-%02d", timeinfo.tm_year + 1900, timeinfo.tm_mon + 1, timeinfo.tm_mday);
4 display.setCursor(0, 16);
5 display.printf("%02d:%02d:%02d", timeinfo.tm_hour, timeinfo.tm_min, timeinfo.tm_sec);
6 display.display();
```

2. Hourly Chime

```
1 if (timeinfo.tm_min == 0 && timeinfo.tm_sec == 0){
2   Serial.println("=== Hour Chime ===");
3   digitalWrite(LED_BUILTIN, HIGH);
4   delay(500);
5   digitalWrite(LED_BUILTIN, LOW);
6 }
```

3. Scheduled Task (e.g., 7 AM daily)

```
1 if (timeinfo.tm_hour == 7 && timeinfo.tm_min == 0 && timeinfo.tm_sec == 0){
2   Serial.println("Good morning!");
3   // Turn on lights, etc.
4 }
```

4. Logging with Timestamps

Add time before all `Serial.print` :

```
1 void logWithTimestamp(const char* msg){
2   time_t now;
3   struct tm ti;
4   time(&now);
5   localtime_r(&now, &ti);
6
7 }
```

```
1 Serial.printf("[%02d:%02d:%02d] %s\n", ti.tm_hour, ti.tm_min, ti.tm_sec, msg);  
}
```

[Back to Elegoo](#)

5. Switch Timezone (at Runtime)

```
1 void setTimeZone(const char* tz) {  
2     setenv("TZ", tz, 1);  
3     tzset();  
4     Serial.printf("[Timezone] Set to: %s\n", tz);  
5 }
```

Notes

Note 1: NTP Sync Depends on UDP Port 123

NTP uses UDP protocol port 123. If your router or firewall blocks UDP 123, ESP32 cannot sync time.

Note 2: Time May Jump During Sync

When NTP syncs, ESP32 may suddenly "jump" time to correct value (e.g., from 1970 to 2026). If your program has special operations during time jumps (like timers), handle this case.

Note 3: `tm_year` and `tm_mon` Corrections

- `tm_year` must `+1900` for actual year
- `tm_mon` must `+1` for actual month
- `tm_wday` needs no correction (0=Sunday, 6=Saturday)

Note 4: ESP32 Must Re-sync After Reboot

ESP32 has no persistent clock, every power-on must repeat WiFi → NTP flow. If your application is sensitive to boot time, add "last sync time" check to skip if recently synced.

Note 5: Timezone String Sign Direction

`CST-8` means UTC+8 (East 8). Sign direction is easy to get wrong, remember the rule: **TZ string offset sign is opposite of "UTC plus how much"**:

- UTC+8 → `-8`
- UTC-5 → `+5`

Note 6: `configTime()` and `setenv()` Order

Must `configTime()` first, then `setenv()` + `tzset()`. If reversed, timezone setting may not take effect.

Note 7: Time Access in Multi-task Environment

If your program gets time in multiple tasks/interrupts, use `localtime_r` (reentrant) not `localtime` (not reentrant).

[← WiFi Stability](#)

[IOT→](#)